

Ejercicio: Lectura y Limpieza de Datos de Café

Emmanuel

2026-05-03

Contents

Actividad 0: Análisis de valores faltantes del objeto <code>datos</code>	1
Actividad 1: Crear la columna <code>color2</code> a partir de <code>color</code>	2
Actividad 2: Crear la columna numérica <code>bag_weight2</code> a partir de <code>bag_weight</code>	3
Actividad 3: Crear <code>method1</code> y <code>method2</code> a partir de <code>processing_method</code>	4
Actividad 4: Crear <code>expiration_day</code> , <code>expiration_month</code> y <code>expiration_year</code>	5
Actividad 5: Crear <code>harvest_mes</code> y <code>harvest_anio</code> a partir de <code>harvest_year</code>	6
Actividad 6: Visualización de <code>total_cup_points</code> , <code>acidity</code> y <code>color2</code>	7
Actividad 7: Visualización de densidad de <code>bag_weight2</code> por <code>species</code>	9
Actividad 8: Fecha de expiración vs <code>total_cup_points</code> (México, Brasil, Colombia, Guatemala) . .	10
Actividad 9: Mes de expiración vs altitudes (México, Brasil, Colombia, Guatemala — 2016 y 2017)	12
Actividad 10: Visualización de <code>aftertaste</code> , <code>acidity</code> , <code>body</code> y <code>species</code> en un mismo canvas	14

```
library(tidyverse)
library(lubridate)
```

```
datos <- readr::read_csv("coffee_ratings.csv")
cat(sprintf("Dimensiones del dataset: %d filas × %d columnas\n", nrow(datos), ncol(datos)))
```

```
## Dimensiones del dataset: 1339 filas × 43 columnas
```

Actividad 0: Análisis de valores faltantes del objeto `datos`

Procedimiento:

Se utilizó `readr::read_csv()` para cargar el dataset, función del paquete `readr` (parte de `tidyverse`) que detecta automáticamente los tipos de columna y retorna un `tibble`. Para el análisis de valores faltantes, las funciones clave son:

- `is.na(datos)`: retorna una matriz lógica (TRUE/FALSE) del mismo tamaño que `datos`, donde TRUE indica un valor faltante.
- `colSums(is.na(datos))`: suma por columna la matriz lógica anterior; como TRUE equivale a 1 y FALSE a 0, el resultado es el conteo de NA por variable.
- `dplyr::filter()`: conserva únicamente las filas que cumplen una condición — aquí, columnas con al menos un valor faltante.
- `dplyr::arrange(desc(...))`: ordena el data frame de mayor a menor según la columna indicada.
- `round()`: redondea el porcentaje a dos decimales.

Resultados:

El dataset contiene **1,339 filas y 43 columnas**. De ellas, **19 presentan valores faltantes**. La columna más afectada es `lot_number` (79.39 %), seguida de `farm_name` (26.81 %) y `mill` (23.75 %). Columnas relevantes

para las actividades siguientes como `color` (20.16 %), `processing_method` (12.70 %) y `harvest_year` (3.51 %) también tienen ausencias.

```
na_counts <- datos |> is.na() |> colSums()

resumen_na <- tibble(
  columna      = na_counts |> names(),
  valores_faltantes = na_counts |> as.integer(),
  porcentaje    = (na_counts / nrow(datos) * 100) |> round(2)
) |>
  filter(valores_faltantes > 0) |>
  arrange(desc(valores_faltantes))

resumen_na |> print(row.names = FALSE)
```

```
## # A tibble: 19 x 3
##   columna      valores_faltantes porcentaje
##   <chr>          <int>         <dbl>
## 1 lot_number      1063          79.4
## 2 farm_name        359          26.8
## 3 mill            315          23.5
## 4 producer        231          17.2
## 5 altitude_low_meters 230          17.2
## 6 altitude_high_meters 230          17.2
## 7 altitude_mean_meters 230          17.2
## 8 altitude        226          16.9
## 9 variety         226          16.9
## 10 color          218          16.3
## 11 company        209          15.6
## 12 processing_method 170          12.7
## 13 ico_number     151          11.3
## 14 region         59           4.41
## 15 harvest_year    47           3.51
## 16 owner           7           0.52
## 17 owner_1         7           0.52
## 18 country_of_origin 1           0.07
## 19 quakers         1           0.07
```

Actividad 1: Crear la columna `color2` a partir de `color`

Procedimiento:

Se creó la columna `color2` mediante el par de funciones más idiomático de `dplyr` para transformaciones condicionales:

- `dplyr::mutate()`: añade o modifica columnas dentro de un `tibble/data.frame` sin alterar las demás. Recibe expresiones que se evalúan columna por columna.
- `dplyr::case_when()`: equivalente vectorizado de múltiples `if/else if/else`. Cada línea es una fórmula condición ~ valor_resultado; la condición `is.na(color) ~ NA_character_` asigna explícitamente NA cuando el color original es faltante (el caso `TRUE ~ NA_character_` actúa como cláusula por defecto).
- `is.na()`: detecta valores NA en un vector.
- `table(..., useNA = "always")`: tabla de frecuencias cruzada que incluye los NA como categoría, útil para verificar el resultado.

Resultados:

La columna `color2` asigna correctamente `#00FF66` a los 870 granos “Green”, `#CCEBC5` a los 114 “Bluish-Green” y `#BFFFFFF` a los 85 “Blue-Green”. Los 270 valores faltantes originales de `color` se conservan como NA.

```
datos <- datos |>
  mutate(
    color2 = case_when(
      is.na(color) ~ NA_character_,
      color == "Green" ~ "#00FF66",
      color == "Bluish-Green" ~ "#CCEBC5",
      color == "Blue-Green" ~ "#BFFFFFF",
      TRUE ~ NA_character_
    )
  )

cat("Distribución de color vs color2:\n")

## Distribución de color vs color2:
print(table(datos$color, datos$color2, useNA = "always"))
```

```
##
##           #00FF66 #BFFFFFF #CCEBC5 <NA>
## Blue-Green         0        85         0    0
## Bluish-Green        0         0       114    0
## Green              870         0         0    0
## None                0         0         0   52
## <NA>                 0         0         0  218
```

Actividad 2: Crear la columna numérica `bag_weight2` a partir de `bag_weight`

Procedimiento:

La columna `bag_weight` almacena cadenas de texto como "60 kg" o "100 lbs". Para extraer solo el valor numérico se emplearon funciones del paquete `stringr`:

- `stringr::str_detect(string, pattern)`: retorna un vector lógico indicando si cada elemento del vector `string` contiene el patrón regex. Se usa para identificar los valores ambiguos que tienen coma (",") indicando múltiples unidades.
- `stringr::str_extract(string, pattern)`: extrae la **primera** subcadena que coincide con el patrón regex. El patrón `"\\d+(\\.\\d+)?"` captura uno o más dígitos seguidos opcionalmente de un punto decimal y más dígitos.
- `dplyr::if_else(condition, true, false)`: versión estricta del `ifelse()` base que respeta los tipos de datos; asigna `NA_real_` (NA de tipo `double`) cuando la condición es verdadera (valor ambiguo).
- `as.numeric()`: convierte la cadena extraída a tipo numérico.

Se consideran **ambiguas** las observaciones con coma en `bag_weight` (p. ej., "2 kg,lbs"), ya que indican múltiples unidades sin un valor único determinable.

Resultados:

Se encontraron **2 observaciones ambiguas**: "1 kg,lbs" y "2 kg,lbs", que reciben NA en `bag_weight2`.

```
ambiguos_bw <- datos |>
  filter(str_detect(bag_weight, ","))

datos <- datos |>
```

```

mutate(
  bag_weight2 = if_else(
    str_detect(bag_weight, ","),
    NA_real_,
    bag_weight |> str_extract("\\d+(\\.\\d+)?") |> as.numeric()
  )
)

vals_ambiguos_bw <- ambiguos_bw$bag_weight |> unique()
cat(sprintf("Observaciones ambiguas en bag_weight: %d\n", nrow(ambiguos_bw)))

## Observaciones ambiguas en bag_weight: 2
cat("Valores ambiguos únicos:", vals_ambiguos_bw, "\n")

## Valores ambiguos únicos: 2 kg,lbs 1 kg,lbs

```

Actividad 3: Crear method1 y method2 a partir de processing_method

Procedimiento:

La columna `processing_method` contiene valores como "Natural / Dry" o "Washed / Wet", separados por "/". La función central es:

- **stringr::str_match(string, pattern)**: aplica el patrón regex al vector `string` y retorna una **matriz** donde la primera columna es la coincidencia completa y las columnas siguientes son los grupos de captura (...). Devuelve NA en filas donde el patrón no coincide, lo que identifica automáticamente los casos ambiguos (valores sin separador o NA).
- El patrón `"^(.+?)\\s*/\\s*(.+)$"` usa dos grupos de captura: el primero `(.+?)` captura el texto antes de "/" de forma no-greedy, y el segundo `(.+)$` captura todo lo que sigue.
- **str_trim()**: elimina espacios al inicio y al final de las cadenas extraídas.
- **sum(is.na(...))**: cuenta cuántas filas no tuvieron coincidencia con el patrón.

Resultados:

El conteo de ambigüedades incluye las 170 filas con NA en `processing_method` más las filas con el valor "Other" (sin separador "/").

```

pm_match <- str_match(datos$processing_method, "^(.+?)\\s*/\\s*(.+)$")

datos <- datos |>
  mutate(
    method1 = pm_match[, 2] |> str_trim(),
    method2 = pm_match[, 3] |> str_trim()
  )

n_ambiguos_pm <- pm_match[, 1] |> is.na() |> sum()
vals_ambiguos_pm <- datos$processing_method[pm_match[, 1] |> is.na()] |> unique()
cat(sprintf("Observaciones ambiguas en processing_method: %d\n", n_ambiguos_pm))

## Observaciones ambiguas en processing_method: 196
cat("Valores únicos ambiguos:", vals_ambiguos_pm, "\n")

## Valores únicos ambiguos: NA Other

```

Actividad 4: Crear `expiration_day`, `expiration_month` y `expiration_year`

Procedimiento:

La columna `expiration` sigue el formato inglés de fecha ordinal: "Month DDth, YYYY" (p. ej., "April 3rd, 2016"). Se utiliza `str_match()` con grupos de captura para extraer los tres componentes en una sola operación:

- **Patrón regex:** `"^((\\w+)\\s+(\\d{1,2})?(?:st|nd|rd|th),\\s+(\\d{4}))$"`
 - `((\\w+))` — grupo 1: nombre del mes (una o más letras/dígitos/guión bajo).
 - `(\\d{1,2})` — grupo 2: día de 1 o 2 dígitos.
 - `(?:st|nd|rd|th)` — sufijo ordinal sin captura (`(?:...)` es un grupo no-capturante).
 - `(\\d{4})` — grupo 3: año de exactamente 4 dígitos.
- `as.integer()`: convierte los grupos de día y año (cadenas) a entero.
- Las filas donde el patrón **no coincide** y `expiration` no es NA se consideran **ambiguas**.

Resultados:

El formato de `expiration` es muy consistente en todo el dataset, por lo que se esperan muy pocas o ninguna observación ambigua.

```
exp_match <- str_match(
  datos$expiration,
  "^((\\w+)\\s+(\\d{1,2})?(?:st|nd|rd|th),\\s+(\\d{4}))$"
)

datos <- datos |>
  mutate(
    expiration_month = exp_match[, 2],
    expiration_day   = exp_match[, 3] |> as.integer(),
    expiration_year  = exp_match[, 4] |> as.integer()
  )

n_ambiguos_exp <- (!is.na(datos$expiration) & is.na(exp_match[, 2])) |> sum()
cat(sprintf("Observaciones ambiguas en expiration: %d\\n", n_ambiguos_exp))
```

```
## Observaciones ambiguas en expiration: 0
```

```
cat("\\nMuestra de las nuevas columnas:\\n")
```

```
##
```

```
## Muestra de las nuevas columnas:
```

```
datos |>
  select(expiration, expiration_day, expiration_month, expiration_year) |>
  head(8) |>
  print()
```

```
## # A tibble: 8 x 4
```

	expiration	expiration_day	expiration_month	expiration_year
	<chr>	<int>	<chr>	<int>
## 1	April 3rd, 2016	3	April	2016
## 2	April 3rd, 2016	3	April	2016
## 3	May 31st, 2011	31	May	2011
## 4	March 25th, 2016	25	March	2016
## 5	April 3rd, 2016	3	April	2016
## 6	September 3rd, 2014	3	September	2014
## 7	September 17th, 2013	17	September	2013
## 8	September 2nd, 2011	2	September	2011

Actividad 5: Crear `harvest_mes` y `harvest_anio` a partir de `harvest_year`

Procedimiento:

La columna `harvest_year` es muy heterogénea: contiene rangos de años ("2016/2017", "2009-2010"), trimestres ("4T/2010"), años simples ("2016") y formatos no estándar ("23 July 2010", "4T72010"). Las funciones clave son:

- `stringr::str_split(string, pattern, n)`: divide cada elemento de `string` en un máximo de `n` partes usando el patrón regex como separador. El resultado es una **lista** donde cada elemento es un vector de carácter. El patrón `"\\s*/\\s*|\\s*-\\s*(?=[0-9])"` usa el operador alternancia `|` para dividir en `"/"` o en `"-"` entre dígitos (lookahead `(?=[0-9])` evita separar en guiones de palabras compuestas).
- `purrr::map_chr(list, function)`: aplica una función a cada elemento de una lista y devuelve un vector de tipo `character`. Es el equivalente vectorizado de `sapply(..., character)` con mayor control de tipos.
- Se considera **ambigua** cualquier fila donde `harvest_anio` resulte NA, lo que indica que no se encontró un separador y el valor no puede dividirse en dos partes.

Resultados:

La principal fuente de ambigüedad son los años simples ("2010", ..., "2018"), los valores malformados ("4T72010") y las fechas completas ("23 July 2010").

```
datos <- datos |>
  mutate(
    partes_hy    = str_split(harvest_year, "\\s*/\\s*|\\s*-\\s*(?=[0-9])", n = 2),
    harvest_mes  = map_chr(partes_hy, ~ str_trim(.x[1])),
    harvest_anio = map_chr(partes_hy, ~ if (length(.x) >= 2) str_trim(.x[2]) else NA_character_)
  ) |>
  select(-partes_hy)

n_ambiguos_hy    <- datos$harvest_anio |> is.na() |> sum()
n_na_originales  <- datos$harvest_year |> is.na() |> sum()
n_sin_separador  <- (is.na(datos$harvest_anio) & !is.na(datos$harvest_year)) |> sum()

cat(sprintf("Observaciones ambiguas en harvest_year: %d\n", n_ambiguos_hy))

## Observaciones ambiguas en harvest_year: 1196

cat(sprintf("  · NaN originales en harvest_year:          %d\n", n_na_originales))

##    · NaN originales en harvest_year:                47

cat(sprintf("  · Valores sin separador divisible:         %d\n", n_sin_separador))

##    · Valores sin separador divisible:                1149

cat("\nValores únicos sin separador (no NA):\n")

##
## Valores únicos sin separador (no NA):

datos$harvest_year[is.na(datos$harvest_anio) & !is.na(datos$harvest_year)] |>
  unique() |>
  sort() |>
  print()
```

```
## [1] "2010" "2011"
## [3] "2012" "2013"
## [5] "2014" "2015"
## [7] "2016" "2017"
## [9] "2018" "23 July 2010"
## [11] "4T72010" "Abril - Julio"
## [13] "August to December" "December 2009-March 2010"
## [15] "Fall 2009" "January 2011"
## [17] "January Through April" "March 2010"
## [19] "May-August" "Mayo a Julio"
## [21] "mmm" "Sept 2009 - April 2010"
## [23] "Spring 2011 in Colombia." "TEST"
```

```
cat("\nMuestra de las nuevas columnas:\n")
```

```
##
```

```
## Muestra de las nuevas columnas:
```

```
datos |>
  select(harvest_year, harvest_mes, harvest_anio) |>
  head(10) |>
  print()
```

```
## # A tibble: 10 x 3
##   harvest_year harvest_mes harvest_anio
##   <chr>         <chr>         <chr>
## 1 2014         2014         <NA>
## 2 2014         2014         <NA>
## 3 <NA>         <NA>         <NA>
## 4 2014         2014         <NA>
## 5 2014         2014         <NA>
## 6 2013         2013         <NA>
## 7 2012         2012         <NA>
## 8 March 2010   March 2010   <NA>
## 9 March 2010   March 2010   <NA>
## 10 2014        2014        <NA>
```

Actividad 6: Visualización de total_cup_points, acidity y color2

Procedimiento:

Se elaboró un diagrama de dispersión con `ggplot2`. Las funciones y capas clave son:

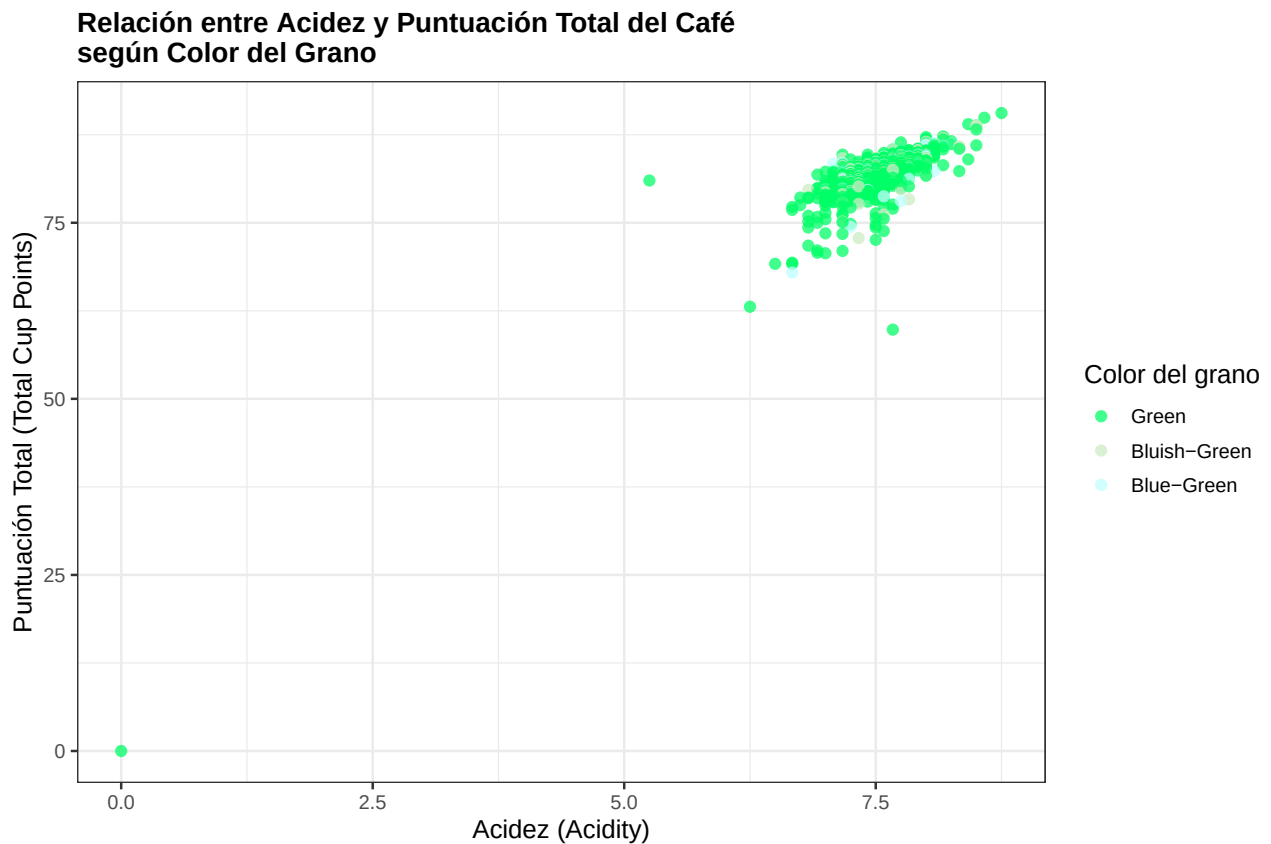
- `ggplot(data, aes(...))`: inicializa el gráfico definiendo los mapeos estéticos globales (`x`, `y`, `color`). La función `aes()` mapea columnas del data frame a propiedades visuales.
- `geom_point()`: capa de puntos para el diagrama de dispersión. Parámetros usados: `alpha` (transparencia), `size` (tamaño), `shape` (forma).
- `scale_color_identity(guide = "legend", ...)`: le indica a `ggplot2` que los valores de la columna `color2` son colores literales (códigos hex), no categorías a mapear a una paleta. Sin esta escala, `ggplot` no sabría interpretar "#00FF66" como un color. El parámetro `guide = "legend"` activa la leyenda; `breaks` y `labels` definen qué valores mostrar y con qué etiquetas.
- `drop_na()`: elimina filas con NA en las columnas indicadas.
- `labs()`: define títulos de ejes, título principal y título de leyenda.
- `theme_bw()`: tema de fondo blanco con cuadrícula gris, adecuado para publicaciones.

Resultados:

El gráfico muestra los tres colores literales del grano. Se aprecia una correlación positiva moderada entre acidez y puntuación total, con poca diferencia entre colores.

```
df6 <- datos |> drop_na(color2, total_cup_points, acidity)

ggplot(df6, aes(x = acidity, y = total_cup_points, color = color2)) +
  geom_point(alpha = 0.75, size = 2.5, shape = 16) +
  scale_color_identity(
    guide = "legend",
    breaks = c("#00FF66", "#CCEBC5", "#BFFFFFFF"),
    labels = c("Green", "Bluish-Green", "Blue-Green")
  ) +
  labs(
    x = "Acidez (Acidity)",
    y = "Puntuación Total (Total Cup Points)",
    title = "Relación entre Acidez y Puntuación Total del Café\nsegún Color del Grano",
    color = "Color del grano"
  ) +
  theme_bw(base_size = 13) +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    legend.position = "right"
  )
)
```



Actividad 7: Visualización de densidad de `bag_weight2` por `species`

Procedimiento:

Se elaboró un gráfico de densidad (estimación por núcleo, KDE) con las siguientes capas y funciones:

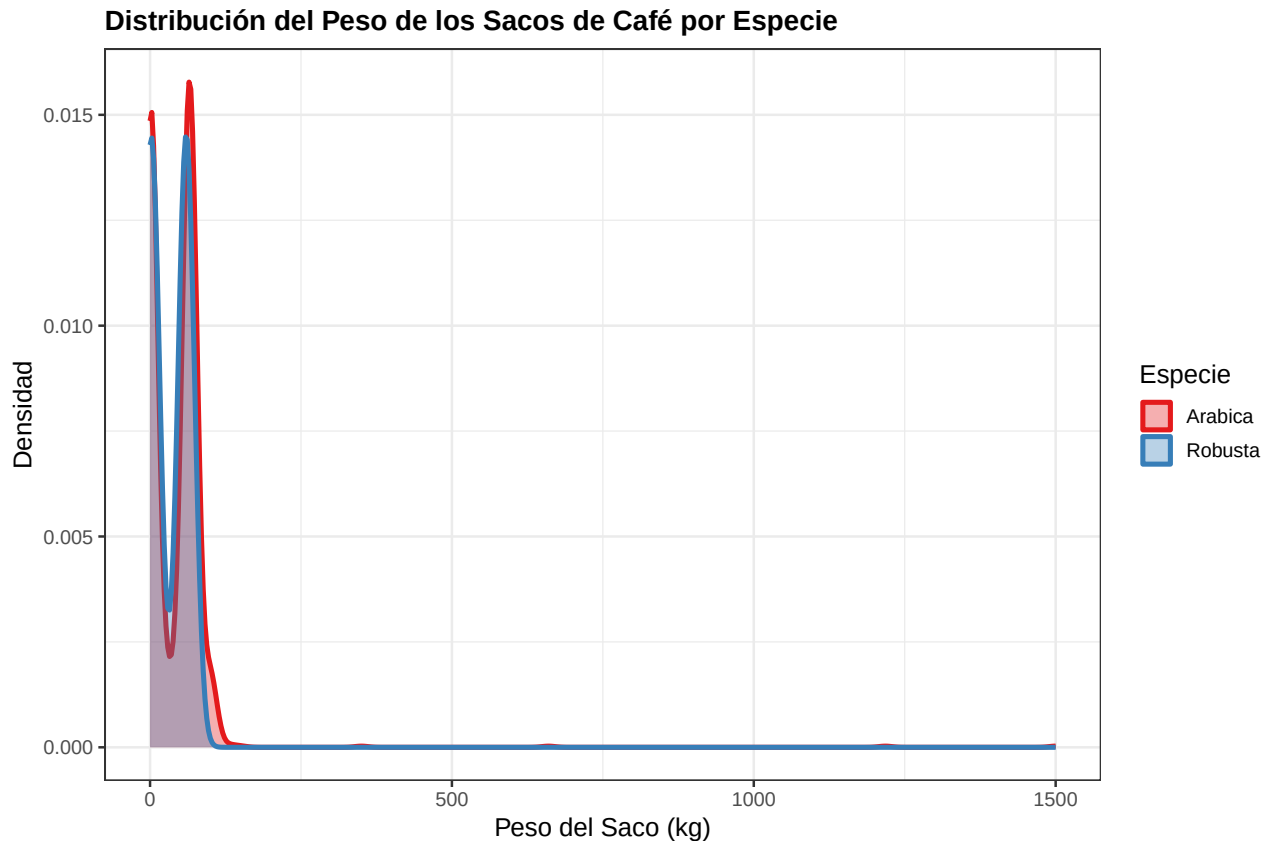
- `geom_density()`: capa que estima y dibuja la función de densidad de probabilidad de la variable continua. Parámetros: `fill` (color de relleno del área bajo la curva), `alpha` (transparencia del relleno), `linewidth` (grosor de la línea de la curva).
- `scale_fill_brewer()` / `scale_color_brewer()`: aplican paletas de color de ColorBrewer (diseñadas para ser perceptualmente distinguibles). Cuando se usan `fill` y `color` a la vez, ambas escalas deben especificarse para que la leyenda sea única.
- `dplyr::filter(bag_weight2 < 2000)`: excluye valores atípicos extremos que distorsionarían la escala horizontal.
- `drop_na()`: elimina filas con NA en las columnas de interés.

Resultados:

La especie *Arabica* (la más frecuente en el dataset) presenta una distribución concentrada en sacos de menor peso, mientras que *Robusta* muestra un patrón diferente con una dispersión menor.

```
df7 <- datos |>
  drop_na(bag_weight2, species) |>
  filter(bag_weight2 < 2000)

ggplot(df7, aes(x = bag_weight2, fill = species, color = species)) +
  geom_density(alpha = 0.35, linewidth = 1.2) +
  scale_fill_brewer(palette = "Set1") +
  scale_color_brewer(palette = "Set1") +
  labs(
    x = "Peso del Saco (kg)",
    y = "Densidad",
    title = "Distribución del Peso de los Sacos de Café por Especie",
    fill = "Especie",
    color = "Especie"
  ) +
  theme_bw(base_size = 13) +
  theme(plot.title = element_text(face = "bold", size = 14))
```



Actividad 8: Fecha de expiración vs total_cup_points (México, Brasil, Colombia, Guatemala)

Procedimiento:

Se construyó una serie de tiempo para los cuatro países. Las funciones importantes son:

- `dplyr::filter(country_of_origin %in% paises)`: el operador `%in%` selecciona filas cuyo valor pertenece al vector `paises`, equivalente a múltiples condiciones `==` unidas por `|`.
- `setNames(1:12, month_levels)`: crea un vector nombrado que mapea nombres de mes (carácter) a su número ordinal; se usa para construir la fecha.
- `lubridate::ymd()`: convierte cadenas en formato "YYYY-MM-DD" a objetos `Date`. Es más robusto que `as.Date()` para formatos no ambiguos.
- `paste(..., sep = "-")`: concatena los componentes año, mes, día en la cadena esperada por `ymd()`.
- `dplyr::group_by() + summarise()`: agrupa las filas por fecha y país, y calcula la media de `total_cup_points` por grupo.
- `geom_line() + geom_point()`: capas para trazar la serie de tiempo con marcadores en cada observación.
- `scale_x_date(date_labels, date_breaks)`: formatea el eje temporal con etiquetas de fecha (`%b %Y` = "Abr 2016") y define el intervalo entre marcas.

Resultados:

El gráfico muestra la puntuación media por fecha de expiración para cada país, revelando tendencias y disponibilidad temporal diferenciada por región.

```

países <- c("Mexico", "Brazil", "Colombia", "Guatemala")

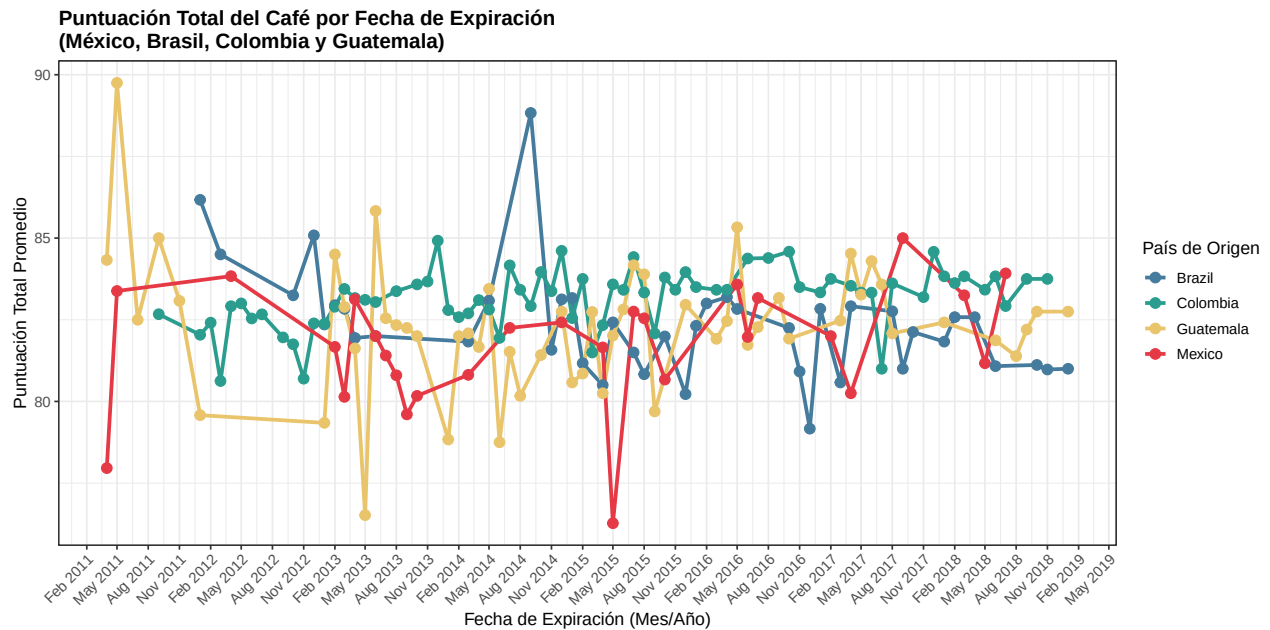
month_levels <- c("January", "February", "March", "April", "May", "June",
                  "July", "August", "September", "October", "November", "December")
month_num <- setNames(1:12, month_levels)

palette_países <- c(
  "Mexico"      = "#E63946",
  "Brazil"      = "#457B9D",
  "Colombia"    = "#2A9D8F",
  "Guatemala"   = "#E9C46A"
)

df8 <- datos |>
  filter(country_of_origin %in% países) |>
  drop_na(expiration_year, expiration_month, total_cup_points) |>
  mutate(
    mes_num = month_num[expiration_month],
    fecha_exp = paste(expiration_year, mes_num, "01", sep = "-") |> ymd()
  ) |>
  group_by(fecha_exp, country_of_origin) |>
  summarise(total_cup_points = mean(total_cup_points, na.rm = TRUE), .groups = "drop") |>
  arrange(fecha_exp)

ggplot(df8, aes(x = fecha_exp, y = total_cup_points,
                color = country_of_origin, group = country_of_origin)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +
  scale_color_manual(values = palette_países) +
  scale_x_date(date_labels = "%b %Y", date_breaks = "3 months") +
  labs(
    x = "Fecha de Expiración (Mes/Año)",
    y = "Puntuación Total Promedio",
    title = "Puntuación Total del Café por Fecha de Expiración\n(México, Brasil, Colombia y Guatemala)",
    color = "País de Origen"
  ) +
  theme_bw(base_size = 12) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(face = "bold", size = 14)
  )

```



Actividad 9: Mes de expiración vs altitudes (México, Brasil, Colombia, Guatemala — 2016 y 2017)

Procedimiento:

Se usó el paradigma *tidy data* para transformar las tres columnas de altitud en filas. Las funciones clave son:

- `tidyr::pivot_longer(cols, names_to, values_to)`: transforma el data frame de formato ancho (*wide*) a largo (*long*), convirtiendo las columnas indicadas en `cols` en dos columnas: `names_to` (nombre de la variable) y `values_to` (su valor). Esto permite graficar las tres altitudes como una sola variable categórica `tipo_altitud`.
- `factor(..., levels = ...)` / `forcats::fct_relevel()`: establece el orden de las categorías del factor `expiration_month` para que el eje horizontal respete el orden cronológico (enero a diciembre) en lugar del orden alfabético.
- `ggplot2::facet_wrap(~ variable)`: divide el gráfico en paneles (facetas) según los valores de `variable`; aquí crea un panel por año de expiración.
- `stat_summary(fun = mean, geom = "line")`: calcula la media de y por grupo y la dibuja como línea, sin necesidad de agregar previamente. Más flexible que `geom_line()` directo cuando hay múltiples observaciones por punto x.
- `scale_color_brewer(palette = "Dark2")`: paleta cualitativa con colores bien diferenciados para las tres altitudes.

Resultados:

La visualización de dos paneles compara cómo varía la altitud de cultivo por mes de expiración en 2016 y 2017, distinguiendo la altitud mínima, media y máxima.

```
month_levels <- c("January", "February", "March", "April", "May", "June",
                  "July", "August", "September", "October", "November", "December")
países <- c("Mexico", "Brazil", "Colombia", "Guatemala")

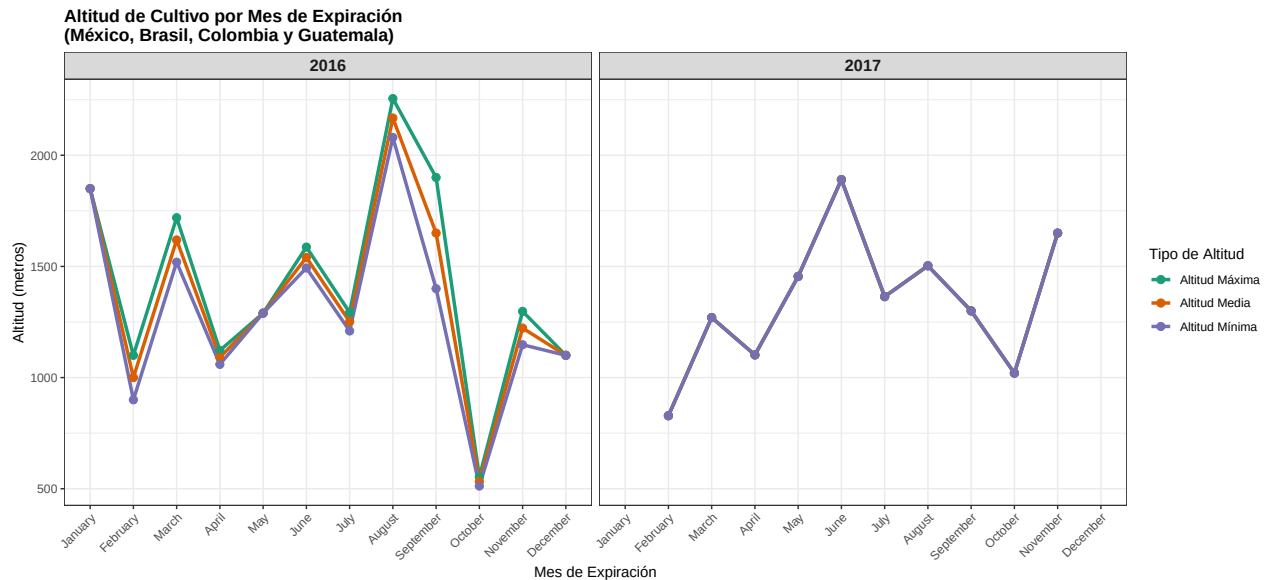
df9 <- datos |>
  filter(
    country_of_origin %in% países,
```

```

    expiration_year %in% c(2016L, 2017L)
) |>
drop_na(expiration_month,
        altitude_low_meters, altitude_mean_meters, altitude_high_meters) |>
mutate(
  expiration_month = factor(expiration_month, levels = month_levels)
) |>
pivot_longer(
  cols      = c(altitude_low_meters, altitude_mean_meters, altitude_high_meters),
  names_to  = "tipo_altitud",
  values_to = "altitud_metros"
) |>
mutate(
  tipo_altitud = case_when(
    tipo_altitud == "altitude_low_meters" ~ "Altitud Mínima",
    tipo_altitud == "altitude_mean_meters" ~ "Altitud Media",
    tipo_altitud == "altitude_high_meters" ~ "Altitud Máxima"
  )
)

ggplot(df9, aes(x = expiration_month, y = altitud_metros,
               color = tipo_altitud, group = tipo_altitud)) +
  stat_summary(fun = mean, geom = "line", linewidth = 1.2) +
  stat_summary(fun = mean, geom = "point", size = 2.5) +
  facet_wrap(~ expiration_year, nrow = 1) +
  scale_color_brewer(palette = "Dark2") +
  labs(
    x      = "Mes de Expiración",
    y      = "Altitud (metros)",
    title  = "Altitud de Cultivo por Mes de Expiración\n(México, Brasil, Colombia y Guatemala)",
    color  = "Tipo de Altitud"
  ) +
  theme_bw(base_size = 11) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title  = element_text(face = "bold", size = 13),
    strip.text   = element_text(face = "bold", size = 12)
  )

```



Actividad 10: Visualización de *aftertaste*, *acidity*, *body* y *species* en un mismo canvas

Procedimiento:

Se usó exclusivamente `tidyverse` (sin paquetes adicionales). La estrategia combina `pivot_longer()` y `facet_wrap()` para mostrar las tres variables en un mismo canvas:

- `tidyr::pivot_longer(cols, names_to, values_to)`: transforma el data frame de formato ancho a largo. Las tres columnas numéricas (*aftertaste*, *acidity*, *body*) se apilan en una sola columna *valor*, y sus nombres pasan a la columna categórica *variable*. Esto permite tratar las tres métricas como facetas de un mismo gráfico.
- `dplyr::case_when()`: renombra los valores de *variable* al español para las etiquetas del gráfico.
- `geom_violin(trim = FALSE)`: dibuja la distribución completa de cada variable para cada especie como una figura simétrica en forma de violín. `trim = FALSE` extiende las colas hasta los valores extremos reales.
- `geom_boxplot()`: superpuesto al violín, muestra la mediana, los cuartiles y los valores atípicos. El argumento `width` controla el ancho del boxplot para que no tape el violín.
- `ggplot2::facet_wrap(~ variable, scales = "free_y")`: divide el canvas en un panel por cada valor de *variable* (uno por métrica). `scales = "free_y"` permite que cada panel tenga su propia escala vertical, ya que las tres variables tienen rangos similares pero distintos.
- `scale_fill_brewer(palette = "Set1")`: aplica una paleta cualitativa distinguible para las especies.

Resultados:

Los tres paneles en un mismo canvas permiten comparar simultáneamente cómo difieren *Arabica* y *Robusta* en *aftertaste*, acidez y cuerpo. *Arabica* concentra sus valores en rangos superiores y con mayor dispersión; *Robusta* muestra distribuciones más compactas y desplazadas hacia valores menores.

```
df10 <- datos |>
  select(aftertaste, acidity, body, species) |>
  drop_na() |>
  pivot_longer(
    cols      = c(aftertaste, acidity, body),
    names_to  = "variable",
    values_to = "valor"
```

```

) |>
mutate(
  variable = case_when(
    variable == "aftertaste" ~ "Aftertaste",
    variable == "acidity"    ~ "Acidez",
    variable == "body"       ~ "Cuerpo"
  )
)

ggplot(df10, aes(x = species, y = valor, fill = species)) +
  geom_violin(alpha = 0.45, trim = FALSE, linewidth = 0.6) +
  geom_boxplot(width = 0.15, alpha = 0.85, outlier.size = 0.8,
    outlier.alpha = 0.5, linewidth = 0.5) +
  facet_wrap(~ variable, scales = "free_y", nrow = 1) +
  scale_fill_brewer(palette = "Set1") +
  labs(
    x    = "Especie",
    y    = "Puntuación",
    title = "Distribución de Aftertaste, Acidez y Cuerpo del Café por Especie",
    fill = "Especie"
  ) +
  theme_bw(base_size = 12) +
  theme(
    plot.title    = element_text(face = "bold", size = 14),
    strip.text    = element_text(face = "bold", size = 12),
    axis.text.x   = element_text(angle = 15, hjust = 1),
    legend.position = "none"
  )
)

```

